

Schema SQL

/* =====

Base transaccional para ventas e inventario

Trabajo en PostgreSQL

===== */

DROP TABLE IF EXISTS historial_envio CASCADE;

DROP TABLE IF EXISTS envio CASCADE;

DROP TABLE IF EXISTS detalle_orden CASCADE;

DROP TABLE IF EXISTS orden CASCADE;

DROP TABLE IF EXISTS inventario CASCADE;

DROP TABLE IF EXISTS producto CASCADE;

DROP TABLE IF EXISTS usuario CASCADE;

/* =====

Tablas principales

===== */

CREATE TABLE usuario (

id_usuario SERIAL PRIMARY KEY,

nombre VARCHAR(100) NOT NULL,

email VARCHAR(150) NOT NULL,

rol VARCHAR(20) NOT NULL CHECK (rol IN ('comprador', 'vendedor', 'admin')),

fecha_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP

);

```
CREATE TABLE producto (  
    id_producto SERIAL PRIMARY KEY,  
    id_vendedor INT NOT NULL,  
    nombre VARCHAR(120) NOT NULL,  
    categoria VARCHAR(80) NOT NULL,  
    precio NUMERIC(12,2) NOT NULL CHECK (precio > 0),  
    activo BOOLEAN NOT NULL DEFAULT TRUE,  
  
    CONSTRAINT fk_producto_vendedor  
        FOREIGN KEY (id_vendedor)  
        REFERENCES usuario(id_usuario)  
);
```

```
CREATE TABLE inventario (  
    id_inventario SERIAL PRIMARY KEY,  
    id_producto INT NOT NULL UNIQUE,  
    stock_actual INT NOT NULL CHECK (stock_actual >= 0),  
    stock_minimo INT NOT NULL CHECK (stock_minimo >= 0),  
    actualizado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
    CONSTRAINT fk_inventario_producto  
        FOREIGN KEY (id_producto)  
        REFERENCES producto(id_producto)  
        ON DELETE CASCADE  
);
```

```
CREATE TABLE orden (  
    id_orden SERIAL PRIMARY KEY,  
    id_comprador INT NOT NULL,  
    id_vendedor INT NOT NULL,  
    fecha_orden TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    estado VARCHAR(20) NOT NULL CHECK (  
        estado IN ('CREADA', 'PAGADA', 'ENVIADA', 'ENTREGADA', 'CANCELADA')  
    ),  
  
    CONSTRAINT fk_orden_comprador  
        FOREIGN KEY (id_comprador)  
        REFERENCES usuario(id_usuario),  
  
    CONSTRAINT fk_orden_vendedor  
        FOREIGN KEY (id_vendedor)  
        REFERENCES usuario(id_usuario)  
);
```

```
CREATE TABLE detalle_orden (  
    id_detalle SERIAL PRIMARY KEY,  
    id_orden INT NOT NULL,  
    id_producto INT NOT NULL,  
    cantidad INT NOT NULL CHECK (cantidad > 0),  
    precio_unitario NUMERIC(12,2) NOT NULL CHECK (precio_unitario > 0),
```

```
CONSTRAINT fk_detalle_orden  
    FOREIGN KEY (id_orden)  
    REFERENCES orden(id_orden)  
    ON DELETE CASCADE,
```

```
CONSTRAINT fk_detalle_producto  
    FOREIGN KEY (id_producto)  
    REFERENCES producto(id_producto)  
);
```

```
CREATE TABLE envio (  
    id_envio SERIAL PRIMARY KEY,  
    id_orden INT NOT NULL UNIQUE,  
    direccion VARCHAR(200) NOT NULL,  
    estado VARCHAR(20) NOT NULL CHECK (  
        estado IN ('PENDIENTE', 'PREPARANDO', 'ENVIADO', 'ENTREGADO', 'CANCELADO')  
    ),  
    fecha_envio TIMESTAMP,
```

```
CONSTRAINT fk_envio_orden  
    FOREIGN KEY (id_orden)  
    REFERENCES orden(id_orden)  
    ON DELETE CASCADE  
);
```

```
CREATE TABLE historial_envio (
```

```

id_historial SERIAL PRIMARY KEY,
id_envio INT NOT NULL,
estado_anterior VARCHAR(20),
estado_nuevo VARCHAR(20) NOT NULL,
fecha_cambio TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
comentario VARCHAR(200),

CONSTRAINT fk_historial_envio
    FOREIGN KEY (id_envio)
    REFERENCES envio(id_envio)
    ON DELETE CASCADE
);

```

```

/* =====
Trigger para actualizar stock
===== */

```

```

CREATE OR REPLACE FUNCTION fn_descontar_stock()
RETURNS TRIGGER AS $$
BEGIN
    /*
    Al registrar un producto dentro de una orden,
    se descuenta la cantidad comprada del inventario.

    La condición stock_actual >= NEW.cantidad evita dejar stock negativo.
    */

```

```

UPDATE inventario

SET stock_actual = stock_actual - NEW.cantidad,

    actualizado_en = CURRENT_TIMESTAMP

WHERE id_producto = NEW.id_producto

    AND stock_actual >= NEW.cantidad;


IF NOT FOUND THEN

    RAISE EXCEPTION

        'No hay stock suficiente para el producto %', NEW.id_producto;

END IF;


RETURN NEW;

END;

$$ LANGUAGE plpgsql;


CREATE TRIGGER trg_descontar_stock

BEFORE INSERT ON detalle_orden

FOR EACH ROW

EXECUTE FUNCTION fn_descontar_stock();


/* =====

Trigger para historial de envíos

===== */


CREATE OR REPLACE FUNCTION fn_guardar_historial_envio()

```

RETURNS TRIGGER AS \$\$

BEGIN

/*

Se guarda el estado inicial del envío.

Luego, cada vez que cambie el estado, se registra el cambio.

*/

IF TG_OP = 'INSERT' THEN

INSERT INTO historial_envio (

id_envio,

estado_anterior,

estado_nuevo,

comentario

)

VALUES (

NEW.id_envio,

NULL,

NEW.estado,

'Registro inicial del envío'

);

ELSIF TG_OP = 'UPDATE'

AND NEW.estado IS DISTINCT FROM OLD.estado THEN

INSERT INTO historial_envio (

id_envio,

estado_anterior,

```

        estado_nuevo,
        comentario
    )
VALUES (
    NEW.id_envio,
    OLD.estado,
    NEW.estado,
    'Cambio de estado del envío'
);
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_historial_envio
AFTER INSERT OR UPDATE OF estado ON envio
FOR EACH ROW
EXECUTE FUNCTION fn_guardar_historial_envio();

/* =====
Índices solicitados
===== */

-- B-Tree para búsquedas de usuario por correo.

```



```
CREATE UNIQUE INDEX idx_usuario_email
```

```
ON usuario USING BTREE (email);
```

```
-- B-Tree compuesto para reportes por comprador y fecha.
```

```
CREATE INDEX idx_orden_comprador_fecha
```

```
ON orden USING BTREE (id_comprador, fecha_orden);
```

```
/* =====
```

```
Datos de prueba
```

```
50 registros mínimos por entidad
```

```
===== */
```

```
-- 50 usuarios: los primeros 10 son vendedores y los demás compradores.
```

```
INSERT INTO usuario (nombre, email, rol, fecha_registro)
```

```
SELECT
```

```
  CASE g
```

```
    WHEN 1 THEN 'Ana Torres'
```

```
    WHEN 2 THEN 'Luis Ramírez'
```

```
    WHEN 3 THEN 'María Gómez'
```

```
    WHEN 4 THEN 'Carlos Rojas'
```

```
    WHEN 5 THEN 'Sofía Herrera'
```

```
    WHEN 6 THEN 'Juan Pérez'
```

```
    WHEN 7 THEN 'Laura Castillo'
```

```
    WHEN 8 THEN 'Diego Vargas'
```

```
    WHEN 9 THEN 'Valentina Mora'
```

```

        WHEN 10 THEN 'Andrés Silva'

        ELSE 'Cliente ' || g

    END,

    'usuario' || g || '@correo.com',

    CASE

        WHEN g <= 10 THEN 'vendedor'

        ELSE 'comprador'

    END,

    CURRENT_TIMESTAMP - (g * INTERVAL '1 day')

FROM generate_series(1, 50) AS s(g);

-- 50 productos repartidos entre los 10 vendedores.

INSERT INTO producto (id_vendedor, nombre, categoria, precio, activo)

SELECT

    ((g - 1) % 10) + 1,

    CASE

        WHEN g <= 10 THEN 'Producto básico ' || g

        WHEN g <= 25 THEN 'Producto estándar ' || g

        WHEN g <= 40 THEN 'Producto premium ' || g

        ELSE 'Producto especial ' || g

    END,

    CASE g % 5

        WHEN 0 THEN 'Tecnología'

        WHEN 1 THEN 'Hogar'

        WHEN 2 THEN 'Ropa'

```

```

        WHEN 3 THEN 'Deportes'

        ELSE 'Libros'

    END,

    (20 + (g * 1.75))::NUMERIC(12,2),

    TRUE

FROM generate_series(1, 50) AS s(g);

-- 50 inventarios, uno por cada producto.

-- Los primeros 15 quedan cerca del stock mínimo para probar el reporte de stock
crítico.

INSERT INTO inventario (id_producto, stock_actual, stock_minimo)

SELECT

    id_producto,

    CASE

        WHEN id_producto <= 15 THEN 18 + (id_producto % 5)

        ELSE 100 + id_producto

    END,

    CASE

        WHEN id_producto <= 15 THEN 20

        ELSE 25

    END

FROM producto;

-- 50 órdenes.

```

```

INSERT INTO orden (id_comprador, id_vendedor, fecha_orden, estado)
SELECT
    11 + ((g - 1) % 40),
    1 + ((g - 1) % 10),
    TIMESTAMP '2026-01-01 09:00:00' + ((g - 1) * INTERVAL '2 days'),
    CASE
        WHEN g % 4 = 0 THEN 'ENTREGADA'
        WHEN g % 3 = 0 THEN 'ENVIADA'
        WHEN g % 2 = 0 THEN 'PAGADA'
        ELSE 'CREADA'
    END
FROM generate_series(1, 50) AS s(g);

```

-- 50 detalles de orden.

-- Aquí se activa automáticamente el trigger que descuenta stock.

```

INSERT INTO detalle_orden (id_orden, id_producto, cantidad, precio_unitario)
SELECT
    o.id_orden,
    ((o.id_orden - 1) % 50) + 1,
    ((o.id_orden - 1) % 5) + 1,
    p.precio
FROM orden o
JOIN producto p
    ON p.id_producto = ((o.id_orden - 1) % 50) + 1;

```

-- 50 envíos.

-- Al insertar estos registros también se llena historial_envio.

INSERT INTO envio (id_orden, direccion, estado, fecha_envio)

SELECT

id_orden,

'Calle ' || (100 + id_orden) || ' # ' || (id_orden % 20) || '-50',

CASE

WHEN id_orden % 4 = 0 THEN 'ENTREGADO'

WHEN id_orden % 3 = 0 THEN 'ENVIADO'

WHEN id_orden % 2 = 0 THEN 'PREPARANDO'

ELSE 'PENDIENTE'

END,

fecha_orden + INTERVAL '2 days'

FROM orden;

-- Cambios extra para demostrar que el historial sí registra trazabilidad.

UPDATE envio

SET estado = 'ENVIADO'

WHERE id_envio BETWEEN 1 AND 10

AND estado IN ('PENDIENTE', 'PREPARANDO');

UPDATE envio

SET estado = 'ENTREGADO'

WHERE id_envio BETWEEN 11 AND 20

AND estado IN ('ENVIADO', 'PREPARANDO');

Query SQL

/* =====

1. Verificación de registros por entidad

===== */

SELECT 'usuario' AS entidad, COUNT(*) AS total FROM usuario

UNION ALL

SELECT 'producto', COUNT(*) FROM producto

UNION ALL

SELECT 'inventario', COUNT(*) FROM inventario

UNION ALL

SELECT 'orden', COUNT(*) FROM orden

UNION ALL

SELECT 'detalle_orden', COUNT(*) FROM detalle_orden

UNION ALL

SELECT 'envio', COUNT(*) FROM envio

UNION ALL

SELECT 'historial_envio', COUNT(*) FROM historial_envio

ORDER BY entidad;

/* =====

2. Verificación de índices B-Tree

===== */

```

SELECT
    tablename,
    indexname,
    indexdef
FROM pg_indexes
WHERE schemaname = 'public'
    AND tablename IN ('usuario', 'orden')
ORDER BY tablename, indexname;

```

```

/* =====

```

3. Prueba del trigger de stock

```

===== */

```

```

WITH stock_inicial AS (
    SELECT
        p.id_producto,
        CASE
            WHEN p.id_producto <= 15 THEN 18 + (p.id_producto % 5)
            ELSE 100 + p.id_producto
        END AS stock_inicial
    FROM producto p
)
SELECT
    p.id_producto,

```

```
p.nombre,  
si.stock_inicial,  
COALESCE(SUM(d.cantidad), 0) AS unidades_vendidas,  
i.stock_actual,  
si.stock_inicial - COALESCE(SUM(d.cantidad), 0) AS stock_esperado,  
CASE  
    WHEN i.stock_actual = si.stock_inicial - COALESCE(SUM(d.cantidad), 0)  
    THEN 'OK'  
    ELSE 'REVISAR'  
END AS validacion  
FROM producto p  
JOIN inventario i  
    ON i.id_producto = p.id_producto  
JOIN stock_inicial si  
    ON si.id_producto = p.id_producto  
LEFT JOIN detalle_orden d  
    ON d.id_producto = p.id_producto  
GROUP BY  
    p.id_producto,  
    p.nombre,  
    si.stock_inicial,  
    i.stock_actual  
ORDER BY p.id_producto  
LIMIT 15;
```



```
/* =====
```

4. Historial general de envíos

```
===== */
```

```
SELECT
```

```
    h.id_historial,
```

```
    h.id_envio,
```

```
    h.estado_anterior,
```

```
    h.estado_nuevo,
```

```
    h.fecha_cambio,
```

```
    h.comentario
```

```
FROM historial_envio h
```

```
ORDER BY h.id_historial
```

```
LIMIT 30;
```

```
/* =====
```

5. Cambios reales de estado en envíos

```
===== */
```

```
SELECT
```

```
    h.id_historial,
```

```
    h.id_envio,
```

```
    h.estado_anterior,
```

```
    h.estado_nuevo,
```

```
    h.fecha_cambio,
```

```
        h.comentario
FROM historial_envio h
WHERE h.estado_anterior IS NOT NULL
ORDER BY h.id_historial;
```

```
/* =====
```

6. Prueba controlada: el trigger no permite stock negativo

```
===== */
```

```
DO $$
```

```
BEGIN
```

```
    BEGIN
```

```
        INSERT INTO detalle_orden (
```

```
            id_orden,
```

```
            id_producto,
```

```
            cantidad,
```

```
            precio_unitario
```

```
        )
```

```
        VALUES (
```

```
            1,
```

```
            1,
```

```
            9999,
```

```
            21.75
```

```
        );
```

```

EXCEPTION WHEN OTHERS THEN

    RAISE NOTICE 'Prueba correcta: no se permitió vender más stock del disponible.
Detalle: %', SQLERRM;

    END;

END;

$$;

```

```

/* =====

```

CONSULTA COMPLEJA 1:

Reporte de ventas mensuales por vendedor

```

===== */

```

SELECT

```

    DATE_TRUNC('month', o.fecha_orden)::DATE AS mes,
    v.id_usuario AS id_vendedor,
    v.nombre AS vendedor,
    COUNT(DISTINCT o.id_orden) AS total_ordenes,
    SUM(d.cantidad) AS unidades_vendidas,
    SUM(d.cantidad * d.precio_unitario)::NUMERIC(12,2) AS total_vendido,
    ROUND(AVG(d.cantidad * d.precio_unitario), 2) AS promedio_por_linea

```

FROM orden o

JOIN usuario v

```

    ON v.id_usuario = o.id_vendedor

```

JOIN detalle_orden d

```

    ON d.id_orden = o.id_orden

```

WHERE o.estado <> 'CANCELADA'

GROUP BY

DATE_TRUNC('month', o.fecha_orden)::DATE,

v.id_usuario,

v.nombre

ORDER BY

mes,

total_vendido DESC;

/* =====

CONSULTA COMPLEJA 2:

Ranking de productos con stock crítico

===== */

WITH ventas_producto AS (

SELECT

d.id_producto,

SUM(d.cantidad) AS unidades_vendidas,

SUM(d.cantidad * d.precio_unitario)::NUMERIC(12,2) AS ingresos_generados

FROM detalle_orden d

JOIN orden o

ON o.id_orden = d.id_orden

WHERE o.estado <> 'CANCELADA'

GROUP BY d.id_producto

),

```

stock_critico AS (
    SELECT
        p.id_producto,
        p.nombre,
        p.categoria,
        i.stock_actual,
        i.stock_minimo,
        COALESCE(vp.unidades_vendidas, 0) AS unidades_vendidas,
        COALESCE(vp.ingresos_generados, 0) AS ingresos_generados,
        i.stock_minimo - i.stock_actual AS deficit_stock
    FROM producto p
    JOIN inventario i
        ON i.id_producto = p.id_producto
    LEFT JOIN ventas_producto vp
        ON vp.id_producto = p.id_producto
    WHERE i.stock_actual <= i.stock_minimo
)
SELECT
    RANK() OVER (
        ORDER BY deficit_stock DESC, unidades_vendidas DESC, ingresos_generados
    DESC
    ) AS posicion,
    id_producto,
    nombre,
    categoria,
    stock_actual,

```

```

stock_minimo,
deficit_stock,
unidades_vendidas,
ingresos_generados
FROM stock_critico
ORDER BY posicion, id_producto;

```

```

/* =====

```

CONSULTA COMPLEJA 3:

Vendedores con ventas superiores al promedio mensual

```

===== */

```

```

WITH ventas_mensuales AS (
    SELECT
        DATE_TRUNC('month', o.fecha_orden)::DATE AS mes,
        v.id_usuario AS id_vendedor,
        v.nombre AS vendedor,
        COUNT(DISTINCT o.id_orden) AS ordenes,
        SUM(d.cantidad * d.precio_unitario)::NUMERIC(12,2) AS total_vendido
    FROM orden o
    JOIN usuario v
        ON v.id_usuario = o.id_vendedor
    JOIN detalle_orden d
        ON d.id_orden = o.id_orden
    WHERE o.estado <> 'CANCELADA'

```

```

GROUP BY

    DATE_TRUNC('month', o.fecha_orden)::DATE,

    v.id_usuario,

    v.nombre
)
SELECT

    vm.mes,

    vm.id_vendedor,

    vm.vendedor,

    vm.ordenes,

    vm.total_vendido,

    (

        SELECT ROUND(AVG(vm2.total_vendido), 2)

        FROM ventas_mensuales vm2

        WHERE vm2.mes = vm.mes

    ) AS promedio_mes,

    ROUND(

        vm.total_vendido - (

            SELECT AVG(vm3.total_vendido)

            FROM ventas_mensuales vm3

            WHERE vm3.mes = vm.mes

        ),

        2

    ) AS diferencia_con_promedio,

    DENSE_RANK() OVER (

        PARTITION BY vm.mes

```

```

ORDER BY vm.total_vendido DESC

) AS ranking_mensual

FROM ventas_mensuales vm

WHERE vm.total_vendido > (

    SELECT AVG(vm4.total_vendido)

    FROM ventas_mensuales vm4

    WHERE vm4.mes = vm.mes

)

ORDER BY

    vm.mes,

    ranking_mensual;

```

Para esta capa transaccional implementé una base de datos en PostgreSQL con tablas relacionadas mediante llaves foráneas y restricciones de integridad. El trigger `trg_descontar_stock` se ejecuta antes de insertar un detalle de orden y descuenta automáticamente la cantidad comprada del inventario, evitando que el stock quede negativo. También implementé el trigger `trg_historial_envio`, que registra el estado inicial de cada envío y guarda cada cambio posterior en `historial_envio`.

Para mejorar el rendimiento, agregué un índice B-Tree sobre `usuario(email)` y un índice compuesto B-Tree sobre `orden(id_comprador, fecha_orden)`. Finalmente, cargué 50 registros mínimos por entidad y desarrollé consultas complejas con JOIN, agregaciones, CTE, subconsultas y funciones de ranking para reportes de ventas mensuales y productos con stock crítico.